
Systeme d'Alarme avec Capteur PIR et Ultrason

Systeme de securite combinant detection de mouvement et mesure de distance

Darix SAMANI SIEWE
Rapport de projet – Systeme a Microcontrolleurs

1^{er} juillet 2026

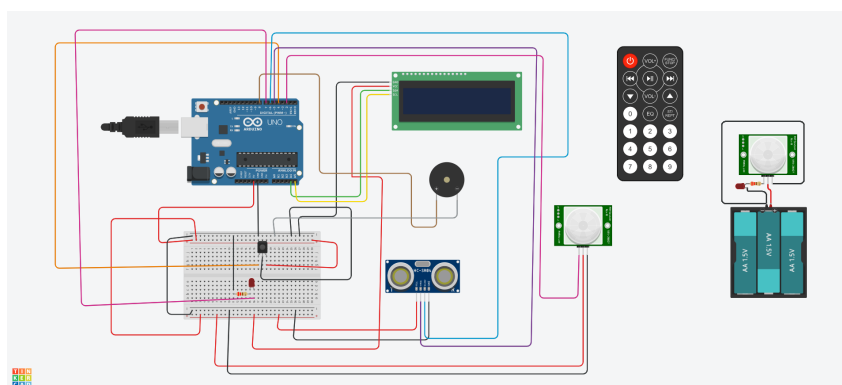


FIGURE 1 – Vue du montage du projet.

Table des matieres

1	Introduction	2
2	Description du Projet	2
3	Composants Utilises	2
3.1	Composants du cahier des charges (version complete)	3
3.2	Composants utilises dans la simulation Tinkercad	3
4	Notions Electroniques Clés	3
5	Schéma du Montage	4
6	Code Arduino	4
7	Analyse et Discussion du Code	7
8	Ressources Complémentaires	8
9	Conclusion	8

1 Introduction

Ce rapport présente la conception d'un **système d'alarme de sécurité** basé sur la carte **Arduino Uno**, combinant plusieurs technologies de détection : un capteur **infrarouge passif (PIR)** pour la détection de mouvement, et un capteur **ultrasonique HC-SR04** pour la mesure de distance. Le système est conçu pour signaler toute intrusion par une combinaison d'alertes visuelles (LED rouge clignotante), sonores (buzzer) et textuelles (affichage sur écran), tout en offrant la possibilité d'être armé ou désarmé à distance via une télécommande infrarouge.

Ce type de projet illustre des notions essentielles de l'électronique embarquée liées à la sécurité : la mesure de distance par temps de vol, la détection pyroélectrique de mouvement, la gestion d'états logiciels (machine à états), et le décodage de protocoles de communication infrarouge.

2 Description du Projet

Le système d'alarme repose sur la complémentarité de deux familles de capteurs :

- le **capteur PIR (HC-SR501)** détecte la présence d'un être vivant dans son champ de vision grâce aux variations de rayonnement infrarouge émis par le corps humain ;
- le **capteur ultrason (HC-SR04)** mesure la distance à un obstacle en chronométrant le temps d'aller-retour d'une onde ultrasonore, permettant de détecter une intrusion par franchissement d'une zone définie.

Lorsqu'une intrusion est détectée (mouvement PIR ou objet trop proche détecté par l'ultrason), le système déclenche une séquence d'alerte :

- une **LED rouge clignote** pour signaler visuellement l'alerte ;
- un **buzzer** émet un signal sonore d'alarme ;
- un **message d'alerte** s'affiche sur l'écran (OLED dans la version cible, LCD I²C dans la simulation Tinkercad).

Une **télécommande infrarouge** permet à l'utilisateur d'armer ou de désarmer le système à distance, évitant ainsi tout déclenchement intempestif lors de l'entrée ou de la sortie légitime d'une zone surveillée. Le système fonctionne donc comme une véritable **machine à états**, avec au minimum trois états : *armé*, *désarmé* et *en alerte*.

Objectif pédagogique : Ce projet permet de combiner plusieurs types de capteurs (numérique tout-ou-rien pour le PIR, mesure de temps pour l'ultrason, décodage de trames pour l'IR) au sein d'une même machine à états, tout en pilotant des actionneurs multiples (LED, buzzer, écran) de façon synchronisée.

3 Composants Utilisés

3.1 Composants du cahier des charges (version complète)

lightgray Composant	Rôle dans le projet
PIR HC-SR501	Détection de mouvement par infrarouge passif
HC-SR04 (x2)	Mesure de distance par écho ultrasonique (zones de surveillance)
Buzzer actif	Émission du signal sonore d'alarme
LED rouge	Signal visuel d'alerte (clignotement)
Écran OLED	Affichage des messages d'état et d'alerte
Récepteur IR	Réception des commandes de la télécommande (armement/désarmement)
Arduino Uno	Microcontrôleur assurant l'acquisition, la logique d'état et le pilotage des sorties

TABLE 1 – Composants prévus dans le cahier des charges initial du projet.

3.2 Composants utilisés dans la simulation Tinkercad

lightgray Réfé- rence	Quantité	Composant
U1	1	Arduino Uno R3
DIST1	1	Capteur de distance ultrasonique (4 broches, type HC-SR04)
U2	1	Écran LCD 16x2 avec module I ² C basé sur le PCF8574 (adresse 0x20)
PIEZO1	1	Buzzer piézoélectrique
U3	1	Capteur/récepteur infrarouge
R1–R8	8	Résistances 220 Ω (limitation de courant des LED)
D1–D8	8	LED rouges
PIR1–PIR8	8	Capteurs PIR (détection de mouvement, zones multiples)
Bat1–Bat7	7	Piles AA 1,5 V (alimentation des modules PIR)

TABLE 2 – Composants effectivement câblés dans la simulation Tinkercad.

Remarque : La simulation Tinkercad utilise un écran **LCD I²C** à la place de l'écran OLED prévu initialement, ainsi qu'un buzzer piézoélectrique générique. Le montage comporte également un réseau de **8 capteurs PIR** et **8 LED**, ce qui suggère une version étendue du système couvrant plusieurs zones de détection simultanées plutôt qu'un capteur PIR unique.

4 Notions Électroniques Clés

- **Mesure de durée par échos ultrasoniques (temps de vol) :** le HC-SR04 émet une impulsion ultrasonique et mesure le temps écoulé jusqu'à la réception de l'écho réfléchi par un obstacle. La distance est obtenue par

$$d = \frac{v_{son} \times \Delta t}{2}$$

où $v_{son} \approx 340$ m/s et Δt est la durée de l'aller-retour de l'onde (division par 2 car l'onde fait un aller-retour).

- **Capteur à infrarouge passif et zones de détection pyroélectrique** : le PIR détecte les variations de rayonnement infrarouge émis par les corps chauds (êtres vivants) dans son champ de vision, grâce à un capteur pyroélectrique sensible aux changements de température dans ses zones de Fresnel.
- **Gestion d'interruptions et machine à états (armé/désarmé/alerte)** : le comportement du système est modélisé par une machine à états finis, où chaque état (armé, désarmé, alerte) détermine la réaction du système aux événements (mouvement détecté, commande IR reçue, distance franchie).
- **Protocole de communication IR (NEC, décodage des trames)** : la télécommande IR transmet des commandes sous forme d'impulsions modulées, encodées selon des protocoles standardisés comme le protocole **NEC**, qui doivent être décodées par le récepteur pour identifier la commande envoyée (armer/désarmer).

5 Schéma du Montage

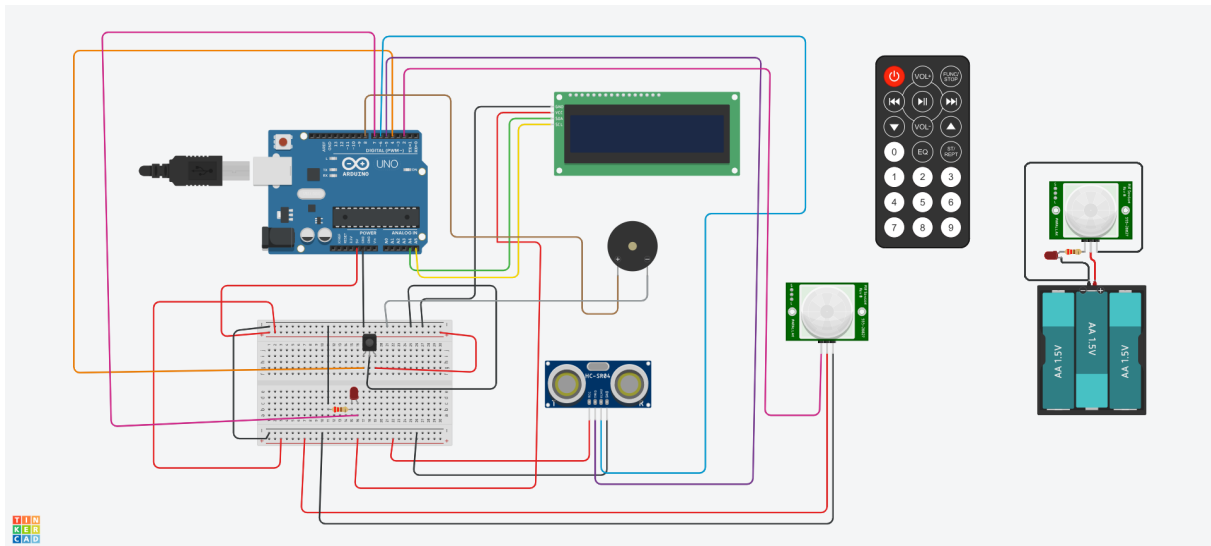


FIGURE 2 – Schéma de câblage du montage (Arduino Uno, capteurs PIR, ultrason, LCD I²C, buzzer, récepteur IR, LED).

6 Code Arduino

Le programme assure l'acquisition des capteurs PIR et ultrason, la gestion de la machine à états (armé/désarmé/alerte), le décodage des commandes de la télécommande IR, ainsi que le pilotage des actionneurs (LED, buzzer, écran).

```

1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3 #include <IRremote.h>
4
5 // --- Configuration de l' écran LCD (Met l'adresse de ton scanner ex: 0x27 ou 0
6   x20)
7 LiquidCrystal_I2C lcd(0x20, 16, 2);

```

```
7
8 // --- Broches Mat rielles ---
9 const int pinIR = 2;
10 const int pinPIR = 4;
11 const int pinTrig = 5;
12 const int pinEcho = 6;
13 const int pinLed = 7;
14 const int pinBuzzer = 8;
15
16 // --- D finition des tats du Syst me ---
17 enum Etats {DESARME, ARME, ALERTE};
18 Etats etatActuel = DESARME;
19
20 // --- Variables de mesure ---
21 const int distanceSeuil = 100; // Seuil d'intrusion en cm (ex: 1 m tre)
22 unsigned long precedentMillis = 0;
23 bool etatLedFlash = false;
24
25 void setup() {
26     Serial.begin(9600);
27
28     // Initialisation des capteurs et actionneurs
29     pinMode(pinPIR, INPUT);
30     pinMode(pinTrig, OUTPUT);
31     pinMode(pinEcho, INPUT);
32     pinMode(pinLed, OUTPUT);
33     pinMode(pinBuzzer, OUTPUT);
34
35     // Initialisation IR et LCD
36     IrReceiver.begin(pinIR, ENABLE_LED_FEEDBACK);
37     lcd.init();
38     lcd.backlight();
39
40     afficherStatut();
41 }
42
43 void loop() {
44     // 1. Gestion des commandes de la t l commande (Armement/D sarmement)
45     if (IrReceiver.decode()) {
46         // Bouton "Power" sur la t l commande Tinkercad
47         if (IrReceiver.decodedIRData.command == 0x00) {
48             if (etatActuel == DESARME) {
49                 etatActuel = ARME;
50                 Bip(2); // Deux bips pour confirmer l'armement
51             } else {
52                 etatActuel = DESARME;
53                 digitalWrite(pinLed, LOW);
54                 noTone(pinBuzzer);
55                 Bip(1); // Un long bip pour le d sarmement
56             }
57             afficherStatut();
58         }
59         IrReceiver.resume();
60     }
61
62     // 2. Ex cution de la logique selon l' tat actuel
63     switch (etatActuel) {
64         case DESARME:
65             // Rien surveiller
66             break;
67
68         case ARME:
69             // Surveillance active : Lecture des capteurs
```

```

70     if (detecterMouvement() && calculerDistance() < distanceSeuil) {
71         etatActuel = ALERTE;
72         afficherStatut();
73     }
74     break;
75
76     case ALERTE:
77         // Comportement en cas d'intrusion : Clignotement et sir ne
78         unsigned long actuelMillis = millis();
79         if (actuelMillis - precedentMillis >= 200) { // Cadence de 200ms
80             precedentMillis = actuelMillis;
81             etatLedFlash = !etatLedFlash;
82
83             if (etatLedFlash) {
84                 digitalWrite(pinLed, HIGH);
85                 tone(pinBuzzer, 880); // Fr quence de la sir ne
86             } else {
87                 digitalWrite(pinLed, LOW);
88                 tone(pinBuzzer, 440);
89             }
90         }
91         break;
92     }
93     delay(50); // Stabilit syst me
94 }
95
96 // --- Fonctions Utilitaires ---
97
98 long calculerDistance() {
99     // Envoi d'une impulsion Trig de 10 microsecondes
100    digitalWrite(pinTrig, LOW);
101    delayMicroseconds(2);
102    digitalWrite(pinTrig, HIGH);
103    delayMicroseconds(10);
104    digitalWrite(pinTrig, LOW);
105
106    // Mesure de la dur e de l' cho (temps de vol en microsecondes)
107    long duree = pulseIn(pinEcho, HIGH);
108
109    // Calcul de la distance en cm : (Vitesse du son 340m/s -> 0.034 cm/us) / 2 (
110    aller-retour)
111    long distance = duree * 0.034 / 2;
112    return distance;
113 }
114
115 bool detecterMouvement() {
116     return digitalRead(pinPIR);
117 }
118
119 void Bip(int nbre) {
120     for (int i = 0; i < nbre; i++) {
121         tone(pinBuzzer, 1000, 100);
122         delay(150);
123     }
124 }
125
126 void afficherStatut() {
127     lcd.clear();
128     lcd.setCursor(0, 0);
129     if (etatActuel == DESARME) {
130         lcd.print("SYSTEME: DISARM");
131         lcd.setCursor(0, 1);
132         lcd.print("Statut: Securise");

```

```

132 }
133 else if (etatActuel == ARME) {
134     lcd.print("SYSTEME: ARME");
135     lcd.setCursor(0, 1);
136     lcd.print("SURVEILLANCE...");
137 }
138 else if (etatActuel == ALERTE) {
139     lcd.print("!! INTRUSION !!");
140     lcd.setCursor(0, 1);
141     lcd.print("ALERTE GENERALE");
142 }
143 }

```

Listing 1 – Code source de la capture d'écran du code source Arduino du système d'alarme.

7 Analyse et Discussion du Code

Sans disposer ici du code source complet, on peut néanmoins décrire la structure logicielle attendue pour un tel système, en s'appuyant sur l'architecture fonctionnelle décrite précédemment :

1. **Initialisation (setup)** : configuration des broches (entrées PIR, broches `trigger`/`echo` de l'ultrason, sortie LED, sortie buzzer), initialisation de l'écran (LCD/OLED) et du récepteur IR, et initialisation de l'état du système (par défaut *désarmé*).
2. **Lecture des capteurs PIR** : lecture numérique (HIGH/LOW) de chaque broche PIR pour détecter une présence. Avec plusieurs capteurs PIR (zones multiples), une boucle ou un tableau de broches permet de scanner l'ensemble des zones à chaque cycle.
3. **Mesure de distance par ultrason** : émission d'une impulsion sur la broche `trigger`, puis mesure de la durée de l'écho sur la broche `echo` via la fonction `pulseIn()`, suivie du calcul de la distance selon la formule du temps de vol.
4. **Décodage de la télécommande IR** : réception et décodage des trames IR (typiquement via une bibliothèque comme `IRremote`) afin d'identifier la commande d'armement ou de désarmement envoyée par l'utilisateur.
5. **Machine à états** : selon l'état courant (*armé*, *désarmé*, *alerte*), le programme décide s'il doit surveiller les capteurs, ignorer leurs lectures, ou déclencher la séquence d'alarme (clignotement LED, activation buzzer, affichage du message d'alerte).
6. **Déclenchement de l'alarme** : lorsque l'état *alerte* est atteint (mouvement PIR détecté ou distance inférieure à un seuil en mode armé), le programme fait clignoter la LED rouge, active le buzzer et affiche un message d'alerte sur l'écran, jusqu'à désarmement par la télécommande.

Points d'attention à intégrer dans le code réel :

- Utiliser `millis()` plutôt que des `delay()` bloquants, afin de pouvoir surveiller en continu les capteurs PIR, l'ultrason et le récepteur IR sans latence excessive.
- Ajouter une temporisation (anti-rebond / délai d'armement) après réception de la commande IR, pour permettre à l'utilisateur de quitter la zone surveillée avant l'activation effective de l'alarme.
- Prévoir une gestion robuste des lectures aberrantes de l'ultrason (timeout sur `pulseIn()` en cas d'absence d'écho).
- Avec 8 capteurs PIR, structurer le code à l'aide d'un tableau de broches et d'une boucle `for` plutôt que de dupliquer le code pour chaque capteur, afin de garder un programme lisible et maintenable.

8 Ressources Complémentaires

- **Vidéo** – *Arduino Security Alarm System : PIR + Ultrasonic Sensor*
<https://www.youtube.com/watch?v=2WMZkiY6kMM>
- **Vidéo** – *Système d'alarme Arduino complet avec buzzer et LED (FR)*
<https://www.youtube.com/watch?v=8hRPrqvmZoE>

9 Conclusion

Ce projet de système d'alarme combine plusieurs principes physiques de détection (infrarouge passif, écho ultrasonique) au sein d'une architecture logicielle organisée autour d'une machine à états. La simulation Tinkercad, en multipliant les capteurs PIR et les LED, illustre une extension naturelle du projet vers une surveillance multi-zones. La version finale, intégrant un écran OLED et un véritable récepteur IR décodant le protocole NEC, permettrait d'obtenir un système de sécurité complet, réactif et contrôlable à distance.